

BigCodeBench/747 Evaluations

Task Description

Count the number of integers and floating-point numbers in a comma-separated string and calculate the sum of their square roots.

- Output: count (int), sqrt_sum (float)
 - Required imports: re, math
-

Model 1 Evaluation

model1_efficiency

Total steps: 5

- Step 1: Problem understanding - **Essential**
- Step 2: Parsing approach using split() and float() - **Essential**
- Step 3: Square root and negative number handling - **Essential**
- Step 4: Implementation details for accumulation - **Essential**
- Step 5: Edge cases and re module discussion - **Essential** (though lengthy, addresses important considerations)

model1_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - logical approach to parsing
- Step 3: Coherent - addresses mathematical constraints
- Step 4: Coherent - implementation details follow from previous steps
- Step 5: Coherent - refines implementation with edge cases

Internal Consistency: No contradictions. Maintains consistent approach throughout.

model1_completeness

Problem Coverage:

- Parse comma-separated string: Yes
- Count integers and floats: Yes
- Calculate square root sum: Yes

- Handle negative numbers: Yes (thoroughly discussed)
- Return count and sqrt_sum: Yes

Factual Correctness: All statements are correct, including math.sqrt behavior

Constraint and edge case handling:

- Negative numbers (counted but not included in sqrt_sum)
 - Whitespace handling
 - Empty strings
 - Invalid conversions
 - Very comprehensive coverage
-

Model 2 Evaluation

model2_efficiency

Total steps: 6

- Step 1: Problem analysis - **Essential**
- Step 2: Processing approach with re.split - **Essential**
- Step 3: Parsing and conversion - **Essential**
- Step 4: Calculation logic - **Essential**
- Step 5: Edge case handling - **Essential**
- Step 6: Return results - **Non-essential** (already implied)

model2_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - chooses re.split approach
- Step 3: Coherent - conversion strategy
- Step 4: Coherent - calculation method
- Step 5: Coherent - error handling
- Step 6: Coherent - conclusion

Internal Consistency: No contradictions identified

model2_completeness

Problem Coverage:

- Parse comma-separated string: Yes
- Count integers and floats: Yes
- Calculate square root sum: Yes
- Handle edge cases: Partially (mentions invalid formats)
- Return count and sqrt_sum: Yes

Factual Correctness: Correct understanding, though less detailed

Constraint and edge case handling:

- Mentions invalid format handling
 - Does not explicitly address negative numbers
-

Model 3 Evaluation

model3_efficiency

Total steps: 7

- Step 1: Problem understanding - **Essential**
- Step 2: String splitting approach - **Essential**
- Step 3: Variable initialization - **Essential**
- Step 4: Iteration and whitespace handling - **Essential**
- Step 5: Number conversion with error handling - **Essential**
- Step 6: Square root calculation for non-negative - **Essential**
- Step 7: Return results - **Essential** (though could be combined with step 6)

model3_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - parsing strategy
- Step 3: Coherent - setup for accumulation
- Step 4: Coherent - preprocessing items
- Step 5: Coherent - validation logic
- Step 6: Coherent - mathematical constraints
- Step 7: Coherent - conclusion

Internal Consistency: Consistent throughout, no contradictions

model3_completeness

Problem Coverage:

- Parse comma-separated string: Yes
- Count integers and floats: Yes
- Calculate square root sum: Yes
- Handle negative numbers: Yes (only non-negative for sqrt)
- Return count and sqrt_sum: Yes

Factual Correctness: All statements are factually correct

Constraint and edge case handling:

- Whitespace handling
 - Empty strings
 - Non-numeric items
 - Negative numbers
 - Good comprehensive coverage
-

Model 4 Evaluation

model4_efficiency

Total steps: 7 (includes implementation)

- Step 1: Split by commas - **Essential**
- Step 2: Strip whitespace - **Essential**
- Step 3: Skip empty tokens - **Essential**
- Step 4: Convert and validate - **Essential**
- Step 5: Count all valid numbers - **Essential**
- Step 6: Square root for non-negative only - **Essential**
- Step 7: Return results - **Essential**

model4_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - preprocessing
- Step 3: Coherent - filtering

- Step 4: Coherent - validation
- Step 5: Coherent - counting logic
- Step 6: Coherent - mathematical constraint
- Step 7: Coherent - output

Internal Consistency: No contradictions, clear implementation provided

model4_completeness

Problem Coverage:

- Parse comma-separated string: Yes
- Count integers and floats: Yes
- Calculate square root sum: Yes
- Handle negative numbers: Yes (explicit)
- Return count and sqrt_sum: Yes
- Includes working code: Yes

Factual Correctness: Correct implementation and logic

Constraint and edge case handling:

- Empty tokens
- Whitespace
- Invalid conversions
- Negative numbers (counted but excluded from sqrt)
- Very clear and complete

Model 5 Evaluation

model5_efficiency

Total steps: 4

- Step 1: Problem understanding and approach - **Essential**
- Step 2: Validation criteria - **Essential** (but mentions incorrect constraint about finiteness/NaN)
- Step 3: Implementation strategy - **Essential**
- Step 4: Final accumulation and return - **Essential**

model5_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - but introduces unnecessary constraints
- Step 3: Coherent - follows from validation logic
- Step 4: Coherent - conclusion

Internal Consistency: Mentions checking for NaN/Infinity which wasn't in requirements

model5_completeness

Problem Coverage:

- Parse comma-separated string: Yes
- Count integers and floats: Partially (adds extra constraints)
- Calculate square root sum: Yes
- Handle negative numbers: Yes
- Return count and sqrt_sum: Yes

Factual Correctness: Incorrectly adds finiteness requirement not in problem

Constraint and edge case handling:

- Mentions whitespace
 - Non-negative handling
 - Adds unnecessary NaN/Infinity checking
-

Model 6 Evaluation

model6_efficiency

Total steps: 5

- Step 1: Problem understanding - **Essential**
- Step 2: Parsing strategy - **Essential**
- Step 3: Square root calculation with constraints - **Essential**
- Step 4: Return format - **Non-essential** (already covered)
- Step 5: Error handling approach - **Essential**

model6_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - parsing approach

- Step 3: Coherent - mathematical constraints
- Step 4: Coherent - output format
- Step 5: Coherent - error handling

Internal Consistency: No contradictions identified

model6_completeness

Problem Coverage:

- Parse comma-separated string: Yes
- Count integers and floats: Yes
- Calculate square root sum: Yes
- Handle negative numbers: Yes
- Return count and sqrt_sum: Yes

Factual Correctness: All statements are correct

Constraint and edge case handling:

- Mentions negative number handling
- Error handling with try-except
- Less detailed than models 1, 3, and 4

Summary

- **Most Complete:** Models 1 and 4 (comprehensive edge case handling, Model 4 includes code)
- **Most Efficient:** Model 5 (only 4 steps, though adds unnecessary constraints)
- **Best Logic:** Models 1, 3, and 4 (clear, coherent progression)
- **Issue Found:** Model 5 incorrectly adds NaN/Infinity checking not in requirements